Madhav Subash Kesavan - 3122245001086 CSE-B Matheswaran S - 3122245001087

UCS3312 – Operating Systems Laboratory

Term Work: Implementation of Subsystems in a Microkernel-based OS

Subsystems Chosen for Implementation

1. Process Manager Subsystem

- Commands Supported: "create_process", "delete_process"
- Functionality: Simulates operating system process creation, deletion, and scheduling.
- Behavior: Launches three child processes in response to the "create_process" command and displays their simulated execution and completion.

2. File System Subsystem

- Commands Supported: "read_file", "write_file"
- Functionality: Simulates file operations by reading from and/or writing to a file (e.g., test.txt).
- Behavior: Reads the contents of test.txt on the "read_file" command; outputs an appropriate error message if the file is missing.

3. Device Driver Subsystem

- Commands Supported: "device_input", "device_output"
- Functionality: Simulates a hardware device (such as a keyboard or monitor).
- Behavior: Prints a message like "Input received: Hello World" when the "device_input" command is received.

IPC Mechanism Explanation

In this project, the communication between the microkernel and its subsystems is achieved using pipes, a standard method of Inter-Process Communication (IPC) in UNIX-like systems.

For each subsystem (Process Manager, File System, Device Driver), the microkernel creates a dedicated pipe prior to forking the corresponding child process. This pipe serves as a unidirectional communication channel from the microkernel to the subsystem.

After the subsystems are forked, the microkernel sends commands such as "create_process", "read_file", or "device_input" through these pipes. Each subsystem reads its command from the pipe, performs the applicable task, and prints output confirming the action.

This approach follows the microkernel design principles where service modules run as separate processes and interact through message passing. Using pipes ensures simple, reliable, and efficient communication while maintaining modular and isolated subsystem management. By avoiding shared memory, it also limits potential faults in one subsystem affecting others.

Overall, pipes provide a clean way to implement the message-passing IPC model fundamental to microkernel architectures, demonstrating modularity, fault-tolerance, and clear separation of concerns between the kernel and subsystems.

Sample Execution Trace

```
gcc microkernel.c -o microkernel

./microkernel

[Kernel] Dispatching tasks to all modules...

[Device Unit] Simulating input from user device...

[File Unit] Reading file: test.txt

[Proc Unit] Launching 3 simulated processes...

[File Unit] Could not open test.txt: No such file or directory

[Proc Unit] Process 1 executing...

[Proc Unit] Process 2 executing...

[Proc Unit] Process 3 executing...

[Device Unit] Input received: Hello World

[Proc Unit] Process 1 done.

[Proc Unit] Process 2 done.

[Proc Unit] Process 3 done.

[Module 0] Shutdown acknowledged.

[Module 1] Shutdown acknowledged.

[Module 2] Shutdown acknowledged.

[Kernel] All modules exited successfully.
```

```
maddi6@selflearn: ~
maddi6@selflearn:~$ gcc microkernel.c -o microkernel
maddi6@selflearn:~$ ./microkernel
[Kernel] Dispatching tasks to all modules...
[Device Unit] Simulating input from user device...
[File Unit] Reading file: test.txt
[Proc Unit] Launching 3 simulated processes...
[File Unit] Could not open test.txt: No such file or directory
[Proc Unit] Process 1 executing...
[Proc Unit] Process 2 executing...
[Proc Unit] Process 3 executing...
[Device Unit] Input received: Hello World
[Proc Unit] Process 1 done.
[Proc Unit] Process 2 done.
[Proc Unit] Process 3 done.
[Module 0] Shutdown acknowledged.
[Module 1] Shutdown acknowledged.
[Module 2] Shutdown acknowledged.
[Kernel] All modules exited successfully.
maddi6@selflearn:~$
```

Learning Outcomes

- Understood the concept and structure of microkernel-based operating systems.
- Learned to implement inter-process communication (IPC) using pipes in C.
- Gained practical skills in modular program design by separating kernel and subsystems.
- Simulated core OS functionalities such as process management, file handling, and device interaction.
- Experienced managing multiple processes and synchronizing subsystem tasks within a microkernel framework.
- Recognized the benefits of modularity and message-passing for robust, maintainable system design.